TransitionList UI設計書

メタデータ

- **コンポーネントID**: TransitionList
- **コンポーネント名**: トランジション一覧
- **ファイルパス**: `frontend/src/components/ExperimentWorker/TransitionList.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

1. 概要

1.1 コンポーネントの目的

選択されたメソッドに含まれるトランジションをテーブル形式で一覧表示し、実験データの記録と問題報告を行うためのコンポーネントです。

1.2 主要機能

- トランジションのテーブル表示
- 検索・フィルタリング機能
- ステータス・進捗率の可視化
- データ記録ボタン
- 問題報告ボタン
- ページング機能

1.3 使用場所

- ExperimentRecordTab（実験記録タブ）のリスト表示モード

2. コンポーネント構成

2.1 画面イメージ


トランジション一覧イメージ

2.2 レイアウト構造

```
┌──────────────────────────────────────────┐
│ 絞り込みUI（Row + Col）                    │
│ ┌──────────────────┬──────────────────┐   │
│ │ [全カラム検索]    │ [トランジション名] │   │
│ │ Input + Search   │ Select           │   │
│ └──────────────────┴──────────────────┘   │
├──────────────────────────────────────────┤
│ テーブル（Table）                          │
│ ┌──────────────────────────────────────┐ │
│ │ トラ|ステ|進捗|材料|条件|結果|記録|問題| │ │
│ │ ンジ|ータ|率  |    |    |    |    |報告| │ │
│ │ ショ|ス  |    |    |    |    |    |    | │ │
│ │ ン名|    |    |    |    |    |    |    | │ │
│ ├──────────────────────────────────────┤ │
│ │ データ行                              │ │
│ │ ...                                  │ │
│ └──────────────────────────────────────┘ │
│ [ページング]                               │
└──────────────────────────────────────────┘

モーダル:
- DataRecordDialog（データ記録）
- IssueManagementModal（問題管理）
```

2.3 使用コンポーネント

Ant Design コンポーネント

- `Table` - トランジション一覧テーブル
- `Input` - 検索入力

- `Select` - トランジション名フィルタ
- `Row`, `Col` - グリッドレイアウト
- `Tag` - ステータス表示
- `Typography.Text` - テキスト表示
- `Progress` - 進捗バー（将来使用予定）
- `message` - 通知メッセージ

カスタムコンポーネント

- `DataRecordDialog` - データ記録モーダル
- `IssueManagementModal` - 問題管理モーダル
- `InstructionConfirmationModal` - 指示確認モーダル（import済み、現在未使用）

アイコン

- `SearchOutlined` - 検索アイコン
- `CheckCircleOutlined` - 完了アイコン
- `ClockCircleOutlined` - 作業中アイコン
- `ExclamationCircleOutlined` - 問題アイコン
- `MinusCircleOutlined` - 未開始アイコン

---

3. Props

```
interface TransitionListProps {
  transitions: Transition[];           // トランジション一覧
  methodName?: string;                 // メソッド名（未使用）
  selectedTaskId?: string;             // 選択中のタスクID
  projectStatus: ProjectStatus | null; // プロジェクトステータス
  onTaskCreated?: (taskId: string) => void; // タスク作成ハンドラ（未使用）
  projectId?: string;                  // プロジェクトID
  methodId?: string;                   // メソッドID
  projectName?: string;                // プロジェクト名（IssueManagementModal・コメント送信・ステータス更新で使用）
```

```
  /** 実験データ保存完了後のコールバック（親コンポーネントの進捗データ再取得に使用） */
  onDataSaveComplete?: () => void;
  /** みなし進捗使用フラグ（organization.use_deemed_progress）: false の場合はチェックボックスを非表示にしてステータス更新を常時表示 */
  useDeemedProgress?: boolean;          // デフォルト: true
}
```

---

4. コントロール一覧

4.1 入力項目

項目名	タイプ	初期値	説明
全カラム検索	Input	''	トランジション名・説明でのあいまい検索
トランジション名フィルタ	Select	undefined	トランジション名での絞り込み

4.2 テーブル列定義

列名	幅	配置	説明
トランジション名	160px	left	固定列、赤文字（#CF0027）で強調
ステータス	100px	center	固定列、Tagで表示
進捗率	65px	center	パーセント表示
材料	55px	center	入力数/総数
条件	55px	center	入力数/総数
結果	55px	center	入力数/総数
記録	65px	center	記録ボタン
問題報告	65px	center	問題報告ボタン
説明	140px	left	説明文（省略可）

4.3 ステータス表示

ステータス	Tag色	アイコン	ラベル

not_started	default	MinusCircleOutlined	未開始
in_progress	processing	ClockCircleOutlined	作業中
completed	success	CheckCircleOutlined	完了
on_hold	warning	ExclamationCircleOutlined	保留中
issue	error	ExclamationCircleOutlined	問題あり

4.4 ボタン

記録ボタン

- **スタイル**: 赤背景（#CF0027）、白文字
- **ホバー**: 濃い赤（#a50020）
- **有効化条件**: `isRecordViewableStatus(projectStatus)` が `true` の場合（waiting_task / executing_task / waiting_approval のいずれか）、または `projectStatus` が `null` の場合
- **無効化時**: グレー背景（#d3d3d3）、opacity: 0.6

問題報告ボタン

- **スタイル**: 白背景、赤枠線（#CF0027）、赤文字
- **ホバー**: 薄ピンク背景（#fff5f7）
- **無効化条件**: なし（常に有効）

4.5 ページング

- **デフォルトページサイズ**: 20件
- **ページサイズ選択肢**: 10, 20, 50, 100件
- **総件数表示**: "全XX件"
- **位置**: 右下（bottomRight）

---

5. イベント

5.1 検索・フィルタイベント

イベント名	トリガー	処理内容
setSearchText	検索入力変更	検索テキストを更新、テーブルを自動フィルタリング
setSelectedTransitionName	トランジション名選択	選択したトランジション名でフィルタリング

5.2 ボタンクリックイベント

イベント名	トリガー	処理内容
handleRecord	記録ボタンクリック	選択したトランジションを設定し、DataRecordDialogを開く
handleIssueReport	問題報告ボタンクリック	選択したトランジションを設定し、関連する問題一覧を取得してIssueManagementModalを開く

5.3 モーダルイベント

イベント名	トリガー	処理内容
handleDataRecordModalClose	データ記録モーダルを閉じる	モーダルを非表示、選択トランジションをクリア
handleDataSave	データ保存完了	成功メッセージ表示、モーダルを閉じ、`onDataSaveComplete` コールバックを呼び出して親コンポーネントの進捗データを再取得
handleIssueModalClose	問題管理モーダルを閉じる	モーダルを非表示、選択トランジションをクリア
handleSubmitIssue	問題報告送信	`createIssue` APIを呼び出して問題を報告し、成功後に問題一覧を再取得して表示を更新
handleSubmitComment	コメント送信	`addComment` APIを呼び出してコメントを投稿し、成功後に問題一覧を再取得して表示を更新
handleUpdateStatus	問題ステータス変更	`updateIssueStatus` APIを呼び出してステータスを更新し、成功後に問題一覧を再取得して表示を更新

---

6. データフロー

6.1 State（内部状態）

```
// 選択中のトランジション
const [selectedTransition, setSelectedTransition] = useState<Transition | null>(null);

// モーダル表示状態
const [instructionModalOpen, setInstructionModalOpen] = useState(false);
const [dataRecordModalOpen, setDataRecordModalOpen] = useState(false);
const [issueModalOpen, setIssueModalOpen] = useState(false);

// 絞り込み用
const [searchText, setSearchText] = useState<string>('');
const [selectedTransitionName, setSelectedTransitionName] = useState<string | undefined>
(undefined);

// 問題一覧
const [issues, setIssues] = useState<Issue[]>([]);

// 問題一覧取得中のローディング状態
const [issueLoading, setIssueLoading] = useState<boolean>(false);
```

6.2 算出データ（useMemo）

```
// トランジション名の選択肢
const transitionNameOptions = useMemo(() => {
  const uniqueNames = Array.from(new Set(transitions.map(t => t.transitionName)));
  return uniqueNames.map(name => ({ label: name, value: name }));
}, [transitions]);

// フィルタリングされたトランジション
const filteredTransitions = useMemo(() => {
  return transitions.filter(transition => {
    // あいまい検索（全カラム対象）
    const matchesSearch = searchText === '' ||
      transition.transitionName.toLowerCase().includes(searchText.toLowerCase()) ||
      (transition.description &&
transition.description.toLowerCase().includes(searchText.toLowerCase()));

    // トランジション名フィルタ
    const matchesTransitionName = !selectedTransitionName ||
      transition.transitionName === selectedTransitionName;

    return matchesSearch && matchesTransitionName;
  });
}, [transitions, searchText, selectedTransitionName]);
```

6.3 使用するAPI

API	メソッド	エンドポイント	用途	実装状況
実験データ保存API	POST	`/api/v1/experiments/projects/:projectId/methods/:methodId/tasks/:taskNo/data`	実験データの一括保存	TODO
ステータス更新API	PUT	`/api/v1/experiments/projects/:projectId/methods/:methodId/tasks/:taskNo/transitions/:transitionId/status`	トランジションステータスの更新	TODO
問題報告API	POST	`/api/v1/issues`	問題報告の送信	✅ 完了
コメント投稿API	POST	`/api/v1/issues/:issueId/comments`	問題へのコメント投稿	✅ 完了
問題一覧取得API	GET	`/api/v1/issues`	問題一覧取得	✅ 完了
問題詳細取得API	GET	`/api/v1/issues/:issueId`	問題詳細とコメント一覧取得	✅ 完了

---

7. スタイリング

7.1 テーブルスタイル

```
scroll={{
  x: 976,                    // 最小幅976px（横スクロール）
  y: 'calc(100vh - 650px)'   // 縦スクロール（動的高さ）
}}
size="middle"
bordered
rowClassName={(_, index) =>
  index % 2 === 0 ? 'table-row-light' : 'table-row-dark'
}
style={{
  background: '#ffffff',
  borderRadius: '8px'
}}
```

7.2 カスタムボタンスタイル

記録ボタン

```
padding: 4px 12px
fontSize: 13px
fontWeight: 500
color: #ffffff
backgroundColor: #CF0027 (通常)
backgroundColor: #a50020 (ホバー)
backgroundColor: #d3d3d3 (無効時)
border: none
borderRadius: 4px
opacity: 0.6 (無効時)
```

問題報告ボタン

```
padding: 4px 12px
fontSize: 13px
fontWeight: 500
color: #CF0027
backgroundColor: #ffffff (通常)
backgroundColor: #fff5f7 (ホバー)
border: 1px solid #CF0027
borderRadius: 4px
```

---

8. バリデーション

8.1 プロジェクトステータスによる制限

```
// プロジェクトの状態が記録を参照・操作可能でない場合は非活性化
// waiting_task / executing_task / waiting_approval の場合に活性
const isProjectRecordable = projectStatus === null || isRecordViewableStatus(projectStatus);
```

8.2 データ表示の処理

```
// 材料・条件・結果のカウント表示
if (total === 0) {
  return <Text type="secondary">-</Text>;
}
const isComplete = input === total;
return (
  <Text strong style={{
    color: isComplete ? '#2e7d32' : '#757575'
  }}>
    {input}/{total}
  </Text>
);
```

---

9. アクセシビリティ

9.1 ARIA属性

```
<Input
  aria-label="全カラム検索"
  // ...
/>

<Select
  aria-label="トランジション名で絞り込み"
  // ...
/>

<Table
  aria-label="トランジション一覧"
  aria-describedby="transition-list-description"
  // ...
/>
```

9.2 ローカライゼーション

```
locale={{
  emptyText: 'トランジションが登録されていません'
}}

pagination={{
  locale: {
    items_per_page: '件/ページ',
    jump_to: 'ページ',
    page: ''
  }
}}
```

---

10. レスポンシブ対応

10.1 グリッド設定

```
<Row gutter={[16, 8]}>
  <Col xs={24} sm={12} md={12}>
    <Input /> // モバイルで全幅、タブレット以上で半幅
  </Col>
  <Col xs={24} sm={12} md={12}>
    <Select /> // モバイルで全幅、タブレット以上で半幅
  </Col>
</Row>
```

10.2 テーブルスクロール

- **横スクロール**: 最小幅976pxで横スクロール有効化
- **縦スクロール**: 動的高さ（calc(100vh - 650px)）

---

11. パフォーマンス考慮事項

11.1 最適化

- `transitionNameOptions` と `filteredTransitions` は `useMemo` でメモ化
- フィルタリングはクライアント側で実行（件数が多い場合はサーバー側フィルタリングを検討）

11.2 データ量

- **想定件数**: 最大100件程度
- **ページサイズ**: デフォルト20件、最大100件
- **横スクロール対応**: 976px以上の幅が必要

---

12. 備考

12.1 特記事項

1. **ステータス表示のカスタマイズ**
   - Tagコンポーネントでアイコン付きステータスを表示
   - 5種類のステータスに対応
2. **プロジェクトステータスによる制御**
   - 記録ボタンは `isProjectRecordable` フラグで制御
   - 問題報告ボタンは常に有効
3. **未使用のProps**
   - `methodName`、`onTaskCreated` は現在未使用
   - 将来の拡張のために残されている
4. **カスタムボタンの使用**
   - Ant DesignのButtonコンポーネントではなく、ネイティブbuttonを使用
   - より細かなスタイル制御のため
5. **IssueManagementModalのデータ管理**

- `IssueManagementModal` は `projectId`、`projectName`、`methodId`、`taskNo`、`transitionId` を受け取り、モーダル内部で自律的にデータ取得・送信を行う
- 問題一覧（`issues`）・ローディング状態（`issueLoading`）はこのコンポーネント内で管理しているが、`IssueManagementModal` へは渡していない
- `handleSubmitIssue`、`handleSubmitComment`、`handleUpdateStatus` もPropsとして `IssueManagementModal` へ渡していない（モーダル内部で処理）

12.2 将来の拡張

1. ソート機能の追加
2. 一括選択・一括操作機能
3. トランジション詳細のインラインプレビュー
4. エクスポート機能（CSV, Excel）
5. カラムの表示/非表示切り替え
6. 保存済みフィルタ設定

12.3 関連ドキュメント

- ExperimentRecordTab UI設計書
- DataRecordDialog UI設計書
- IssueManagementModal UI設計書
- Transition型定義

---

13. 変更履歴

バージョン	日付	変更内容	担当者
1.2.0	2026-03-11	ソースコード実態に合わせた仕様書修正	Cline

		- Props に `projectName`、`onDataSaveComplete`、`useDeemedProgress` を追記	
		- `selectedTaskId` の「未使用」注釈を削除	
		- カスタムコンポーネントに `InstructionConfirmationModal` を追記（import済み・未使用）	
		- State に `issueLoading` を追記	
		- `handleDataSave`、`handleSubmitIssue`、`handleSubmitComment` のTODO記載を実装済みに更新	
		- `handleUpdateStatus`（問題ステータス変更ハンドラ）を追加	
		- 記録ボタン有効化条件を `isRecordViewableStatus` に修正	
		- バリデーション（8.1節）のコードを実装に合わせて更新	
		- IssueManagementModalのデータ管理に関する特記事項を追加	
		- 未使用Propsの記載を修正（`methodName`、`onTaskCreated` に限定）	
1.1.0	2026-02-17	API仕様v2.1.0に準拠した修正	Cline
		- API一覧にエンドポイント列を追加	
		- 実験データ保存APIのエンドポイント更新	
		- ステータス更新APIのエンドポイント更新	
		- 重複していた問題報告APIの削除	
1.0.0	2026-01-19	初版作成	Cline